

Informatika pro moderní fyziky (1) základy automatizace; jednoduché zpracování a vizualizace dat

František HAVLŮJ

e-mail: haf@ujv.cz

ÚJV Řež – oddělení Reaktorové fyziky a podpory palivového cyklu

zimní semestr 2026/2027 – 24. září 2026

1 Úvod

- Syllabus semináře
- K čemu je počítač?
- Problém č. 1: vykreslování dat z detektoru
- Problém č. 2: jehla v kupce sena
- Problém č. 3: zmatek v laborce
- Problém č. 4: mnoho výpočtů, inženýrova smrt

2 Odbočka: správa zdrojového kódu

3 Ruční a poloautomatická řešení

- Problém č. 1: rozbor situace
- Řešení
- Zhodnocení
- Problém č. 2, 3, 4: jak na to?

4 Skriptovací jazyky

- Úvod do skriptování
- Úvod do jazyka Ruby
- První kroky s Ruby

Obsah

1 Úvod

- Sylabus semináře
- K čemu je počítač?
- Problém č. 1: vykreslování dat z detektoru
- Problém č. 2: jehla v kupce sena
- Problém č. 3: zmatek v laborce
- Problém č. 4: mnoho výpočtů, inženýrova smrt

2 Odbočka: správa zdrojového kódu

3 Ruční a poloautomatická řešení

4 Skriptovací jazyky

Profil absolventa předmětu

- chápe počítač nikoli jako psací stroj anebo černou skříňku pro specializované aplikace, ale především jako flexibilní a vysoce univerzální nástroj pro každodenní úkoly ve zpracování dat, jejich prezentaci a tvorbě dokumentů
- orientuje se v moderních paradigmatech praktické informatiky, programování na úrovni běžných skriptů je pro něj samozřejmostí a díky solidnímu přehledu je schopen se v daném problému zorientovat a vybrat si pro jeho řešení vhodný nástroj
- není nucen vykonávat mechanickou a nudnou činnost, ale úkoly řeší kreativně - raději si ad hoc vytvoří na míru šitý skript, který jej ochrání před lidskou chybou i frustrací z monotónnosti

Průběh výuky

- zadání praktické úlohy, jejíž vyřešení je motivací pro obsah lekce
- přednáška na probírané téma, poskytující jak teoretický základ, tak přehled konkrétních nástrojů a postupů
- samostatná práce na řešení daného problému
- společná diskuse nad jednotlivými řešeními a jejich zhodnocení

Organizačně

- čtvrtek 14:00-16:30 bez organizované přestávky (každý dle vlastních potřeb)
- předpokládá se aktivní účast (v uvozovkách povinná)
- v případě absence: vyžaduju po vás, abyste si **doplnili vše z lekce a měli hotové to, co ostatní** – lekce na sebe většinou navazují!
- těm, kdo celý semestr chodili, zadám v posledním týdnu před Vánoci zápočtovou úlohu

K čemu je počítač?

- počítače udělají cokoli, pokud na to existuje postup
- pokud na něco existuje postup, není na to potřeba člověk
- existuje-li postup, existuje také algoritmus
- kdo má algoritmus, může napsat program



Proč se zabývat automatizací?

- mechanická práce je otravná
- program neudělá (náhodnou) chybu
- skript trvá stejně dlouho pro libovolný objem dat
- pokud je potřeba něco pozměnit nebo jen zpracování zopakovat, je ruční práce vepsí



Má cenu se to učit v době AI?

- zaprvé nevím
- zadruhé já sám dělám aktuálně tak 90% v Claude Code
- zatřetí už jsem kvůli tomu zničil spoustu věcí
- začtvrté si myslím, že je stejně potřeba to umět, i když si to s AI hodně ulehčíme

Jak AI v tomto předmětu používat a nepoužívat – návrh

- ptejte se ho na co chcete
- nechte si poradit jak se má něco konkrétního udělat
- zkuste mu neposílat celý svůj kód, aby ho upravil
- nenechte si kód napsat

Zadání

1

Na konci provozní směny je potřeba vyhodnotit signály ze čtyř detektorů a vykreslit je do grafu (signál v závislosti na čase). Data dostáváte v jednoduchém textovém souboru (dva sloupce, spousta řádků). Je potřeba vykreslit do jednoho grafu všechny čtyři detektory. Potíž je, že taková data přicházejí každý den - tento úkol je tedy potřeba řešit opakovaně.

S hvězdičkou

Počet detektorů je proměnný (1 až 9).

Úvod

Odbočka: správa zdrojového kódu
Ruční a poloaautomatická řešení
Skriptovací jazyky

Sylabus semináře

K čemu je počítač?

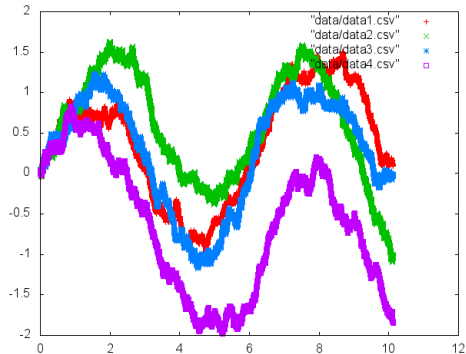
Problém č. 1: vykreslování dat z detektoru

Problém č. 2: jehla v kupce sena

Problém č. 3: zmatek v laborce

Problém č. 4: mnoho výpočtů, inženýrova smrt

Příklad



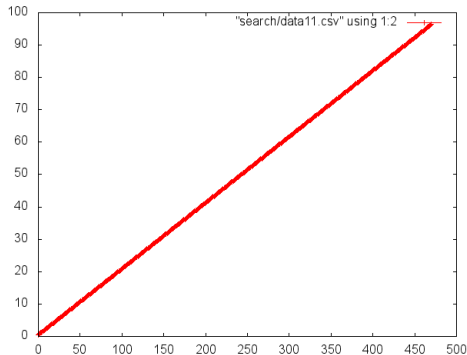
Zadání

2

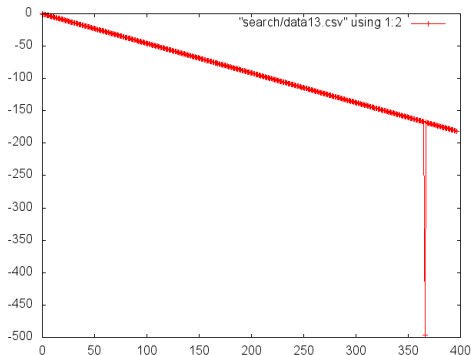
Adresář plný CSV souborů (stovky souborů) obsahuje data, která jsou záznamy signálů s lineární závislostí.

V pěti z nich jsou ale poruchy - data ležící zcela mimo přímku. Kde?

Příklad - dobrý signál



Příklad - špatný signál



Zadání

3

V jedné tabulce máme seznam vzorků půdy, jejich datum pořízení a místo odběru, v jiné změřenou aktivitu z laboratoře. To všechno musíme dostat dohromady a dopočítat se původní aktivity vzorků v době jejich odběru.

Do toho je potřeba výsledky nějak dostat do excelu. Navíc bychom se možná rádi trochu předvedli a vzorky a jejich aktivity vykreslili do mapy.

Zadání

4

Při přípravě základního kritického experimentu je pomocí MCNP potřeba najít kritickou polohu regulační tyče R2. Jak se tato poloha změní při změně polohy tyče R1?

Obsah

- 1 Úvod
- 2 **Odbočka: správa zdrojového kódu**
- 3 Ruční a poloautomatická řešení
- 4 Skriptovací jazyky

Jak mít uložený zdrojový kód

- to, že mám na disku hromadu souborů a 'jen tak' je edituju, přináší spoustu problémů
- pokud udělám chybu/něco se rozbije, nemám se jak vrátit, nemám přehled o průběhu změn
- pokud chci mít víc verzí najednou (stabilní / vývojová), nemám jak zajistit konzistenci
- není jak se vracet ke starším verzím, problémy de facto nelze reprodukovat

Práce v týmu

- pokud na projektu pracuje víc lidí, stává se z problému noční můra
- jak dát dohromady změny od více lidí?
- jak zajistit, že všichni používají stejnou verzi?
- pro dokumenty už jsou různá více či méně dobrá řešení (slučování změn v MS Office, online nástroje jako Google Documents)

Jaké jsou možnosti

- nabízí se schovávat si každý týden verzi a pokusit se v tom udržet pořádek
- dbát na správná jména složek/souborů
- za žádnou cenu nemodifikovat verzovanou složku (read-only...)
- vést záznam o změnách, všechno podrobně dokumentovat

Jak se to dělá správně

- existují takzvané version control systems – VCS, které se o řešení tohoto problému postarají za nás
- základní princip: neukládám aktuální verze souborů, ale pouze *změny*
- k čemu to je – změny je možné mezi sebou **sčítat**, což je (po lehkém zamyšlení) naprosto geniální a řeší to všechno
- pokud mám paralelní změny (dva různí lidé něco změní), není to obvykle vůbec žádný problém, pokud nezměnili stejné místo
- můžu rekonstruovat libovolný bod v historii

Řešení konfliktů – merging

- pokud dva uživatelé udělají změnu zároveň, co se stane?
- když konflikt nenastal na stejném řádku stejného souboru, tak je to v pořádku
- v opačném případě to prostě musí vyřešit uživatel ručně (ale není to častá situace)
- spojení změn se ale vždycky musí dělat lokálně (na straně uživatele), aby se zamezilo sémantickému konfliktu

Paralelní větve – branches

- často se hodí mít víc verzí najednou
- jednak různé stupně experimentálnosti (stabilní, testovací, vývojová)
- jednak pokud chci přidat větší feature, tak to nechci míchat s hlavní větví vývoje
- VCS to většinou umí řešit – strom vývoje nemusí být lineární, ale může se větvit

Přehled VCS

- CVS – první standardní VCS, dnes již zcela překonaný
- SVN – moderní centralizovaný VCS, donedávna nejpoužívanější řešení
- Git – dnešní de facto standard, pokročilý distribuovaný VCS s geniální podporou branches
- Mercurial, Bazaar – méně obvyklé alternativy Gitu

Centralizovaný vs. distribuovaný VCS

- centralizovaný: mám jeden centrální repozitář, do kterého ukládají změny všichni uživatelé
- distribuovaný: každý uživatel má celý repozitář u sebe
- výhody – lokální commity (bez spojení se serverem!), lokální branchy, bezpečnost (vysoká redundance)
- nevýhody – někomu to dělá problém pochopit a přijmout

Kdy je dobré používat VCS

- vlastně vždycky, pokud pracuju s plaintextovými soubory (tj. zejména zdrojové kódy, skripty)
- využití pro binární data (což se týká de facto i wordovského dokumentu...) je velice omezené
- – další důvod používat LaTeX! tam je samozřejmě všechno v pořádku
- výhody verzování (oproti tomu to “jen někde mít”) jsou nedožírné
- za sebe můžu říct, že asi úplně všechnu svoji práci mám v nějakém repo
- a pro tu AI práci je to vlastně dneska úplně nezbytnost

Ukázka a příklad veřejných repozitářů

- `http://github.com`
- `https://github.com/weshatheleopard/rubyXL`
- vhodné zejména pro open source projekty, ale umožňuje i soukromé repo
- velmi hezké GUI (přehledy změn, grafy větvení, issue tracker, komunikace mezi vývojáři)
- samozřejmě není problém si nainstalovat repozitář (např. `gitolite`) někde u sebe na serveru

Repozitář IMF

- přejděte na `https://support.orf.ujv.cz/imf` – to by mělo jít bez registrace
- v projektu **IMF 2026-2027** byste měli vidět vše pro dnešní den
- nainstalujte si git `https://git-scm.com/install/windows`
- stáhněte si repozitář: `git clone https://support.orf.ujv.cz/imf/imf-2026-2027.git`

Váš vlastní repozitář

- na našem ÚJV gitlabu se zaregistrujte
- vytvořte si prázdný repozitář a vyklonujte si ho
- a tam pracujte

Obsah

- 1 Úvod
- 2 Odbočka: správa zdrojového kódu
- 3 Ruční a poloautomatická řešení**
 - Problém č. 1: rozbor situace
 - Řešení
 - Zhodnocení
 - Problém č. 2, 3, 4: jak na to?
- 4 Skriptovací jazyky

Vstupní data

```
0.000000e+00 0.000000e+00
1.000000e-04 1.01447e-03
2.000000e-04 4.62446e-04
3.000000e-04 6.92465e-04
4.000000e-04 4.48142e-03
5.000000e-04 6.95896e-03
6.000000e-04 5.12501e-03
7.000000e-04 2.62076e-03
...
```

Formát CSV

- comma separated values
- zobecnělo ale jako libovolný formát po sloupcích uložených dat
- dobře se zpracovává, importuje do Excelu atd.

Klasické řešení (MS Excel)

- jaké všechny kroky by bylo potřeba udělat?
- na který z provedených kroků byl potřeba člověk – co z toho by nemohl stejně dobře udělat počítač sám?
- jaké jsou výhody a nevýhody ručního řešení?

Co by takové automatické řešení mohlo umět?

- načte z daného adresáře soubory se záznamy
- vykreslí graf a uloží ho do souboru
- soubor jednoznačně pojmenuje a zkopíruje na vhodné místo
- uživatel by neměl v ideálním případě dělat vůbec nic

Komponenty pro automatizaci

Funkční části

výkonné programy (např. kreslení grafů, generování tabulek/reportů, spouštění výpočtů) – předpokladem je možnost spouštět program v neinteraktivním (dávkovém) režimu

Komponenty pro automatizaci

Funkční části

výkonné programy (např. kreslení grafů, generování tabulek/reportů, spouštění výpočtů) – předpokladem je možnost spouštět program v neinteraktivním (dávkovém) režimu

Jak to slepit dohromady

dávkový soubor (BAT) nebo skript – je nutno vždy vhodně volit použité prostředky ve vztahu k jednoduchosti, požadavkům na funkce, přenositelnosti

Jak postupovat s automatickým řešením?

- 1 vykreslit graf s jedním detektorem

Jak postupovat s automatickým řešením?

- 1 vykreslit graf s jedním detektorem
- 2 se všemi detektory

Jak postupovat s automatickým řešením?

- 1 vykreslit graf s jedním detektorem
- 2 se všemi detektory
- 3 z příkazové řádky

Jak postupovat s automatickým řešením?

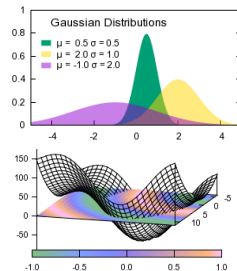
- 1 vykreslit graf s jedním detektorem
- 2 se všemi detektory
- 3 z příkazové řádky
- 4 z batch souboru

Jak postupovat s automatickým řešením?

- 1 vykreslit graf s jedním detektorem
- 2 se všemi detektory
- 3 z příkazové řádky
- 4 z batch souboru
- 5 se jménem adresáře jako parametrem

Gnuplot

- interaktivní i dávkový režim – ideální pro automatizaci
- slušně konfigurovatelné 2D i 3D grafy
- i bez nastavení funguje velmi příjemně
- široká paleta výstupních formátů

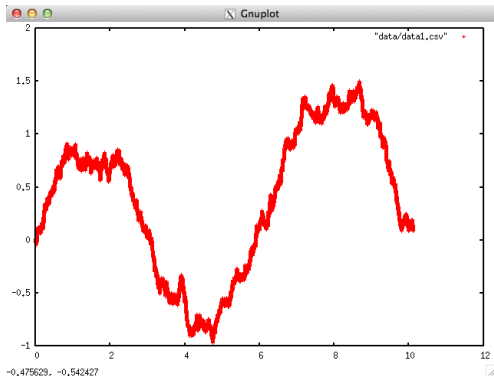


Vykreslení jednoho grafu v gnuplotu

```
gnuplot> plot "data1.csv"
```

Vykreslení jednoho grafu v gnuplotu

```
gnuplot> plot "data1.csv"
```

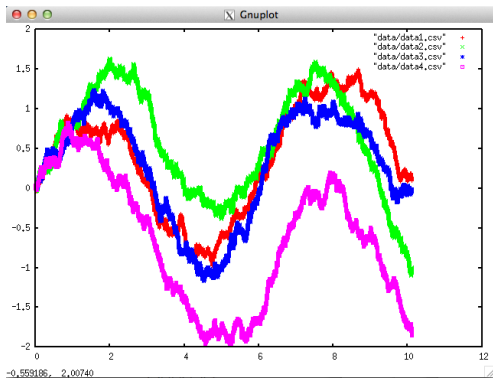


Vykreslení všech grafů v gnuplotu

```
gnuplot> plot "data1.csv", "data2.csv", \  
             "data3.csv", "data4.csv"
```

Vykreslení všech grafů v gnuplotu

```
gnuplot> plot "data1.csv", "data2.csv", \  
              "data3.csv", "data4.csv"
```

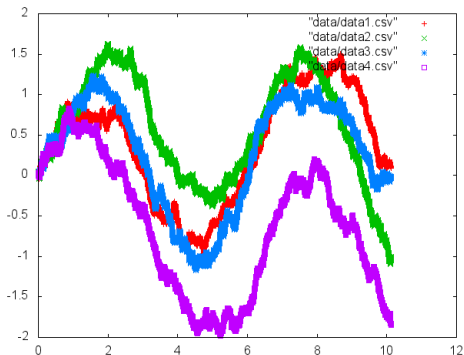


Dávkové použití

```
set terminal png
set output "plot4.png"
plot "gnuplot/data1.csv", "gnuplot/data2.csv", \
     "gnuplot/data3.csv", "gnuplot/data4.csv"
```

Dávkové použití

```
set terminal png  
set output "plot4.png"  
plot "gnuplot/data1.csv", "gnuplot/data2.csv", \  
      "gnuplot/data3.csv", "gnuplot/data4.csv"
```



BAT soubor (Windows)

Je pracné pokaždé vypisovat parametry na příkazovou řádku.
.BAT soubory ve Windows fungují jednoduše, prostě se do nich dá psát jako do terminálu a připravit si tak jednodušší skript.

```
"C:/Program Files/Gnuplot/bin/gnuplot.exe" plot.gp
```

(pozor na uvozovky kvůli mezeře v cestě. Cestu upravte podle skutečnosti na svém počítači.)

Shell skript (MacOS, Linux)

Skript bude trochu jednodušší a velmi podobný:

```
gnuplot plot.gp
```

Uložte ho (konvence) s koncovkou `.sh`. Pozor, ještě je nutno jej změnit na spustitelný pomocí `chmod`, tedy pokud se jmenuje `plot.sh`, použijte:

```
chmod +x plot.sh
```

– to stačí udělat jednou, pak ho můžete libovolně editovat.
Skript se pak z terminálu spustí:

```
./plot.sh
```

BAT soubor s parametrem

Co takhle adresář pro každý den? Vizte adresář 'multidata'.
Nemá smysl pokaždé ručně kopírovat vstup pro gnuplot a tak dále...

Stačí vědět, že BAT soubor může mít na příkazové řádce parametry. První parametr je uložen do proměnné %1 a to se nám bude hodit.

```
cd %1  
"C:/Program Files/Gnuplot/bin/gnuplot.exe" ../plot.gp  
cd ..
```

Opět alternativa pro shell skript

Proměnné se označují místo procenta dolarem, tj. \$1:

```
cd $1  
gnuplot ../plot.gp  
cd ..
```

BAT soubor s parametrem - vylepšení

Pokud si budeme chtít prohlédnout grafy, bude nutné vždy vlézt do adresáře a otevřít `plot.png`. Jde to ovšem vylepšit pomocí jednoduchého triku:

```
cd %1
"C:/Program Files/Gnuplot/bin/gnuplot.exe" ../plot.gp
copy plot.png ..\%1.png
cd ..
```

Totéž v shellu

```
cd $1  
gnuplot ../plot.gp  
cp plot.png ../$1.png  
cd ..
```

Jak moc jsme si pomohli?

- jeden skript místo excelovské anabáze
- máme znovupoužitelný nástroj – můžeme proces kdykoliv zopakovat
- nelze udělat žádnou “ruční chybu”
- skript můžeme dát kolegovi a ten má práci hotovou úplně zadarmo
- skript lze periodicky spouštět bez účasti uživatele, možnost např. zobrazovat na intranetu aktuální grafy atd.

Stačí nám to na řešení problému č. 2, 3 a 4?

Zatím nevíme, jak:

- prohledat adresář a vygenerovat spoustu grafů
- vygenerovat vstupní soubory pro MCNP
- spustit hromadu MCNP výpočtů
- vytahat výsledky z MCNP výstupního souboru
- propojovat data z více tabulek (i když se to dá v excelu naučit)
- získávat a vykreslovat geografická data

Budeme potřebovat nějaký těžší kalibr.

Obsah

- 1 Úvod
- 2 Odbočka: správa zdrojového kódu
- 3 Ruční a poloautomatická řešení
- 4 **Skriptovací jazyky**
 - Úvod do skriptování
 - Úvod do jazyka Ruby
 - První kroky s Ruby

“klasické” programování – Pascal, C++

- napsat zdroják, zkompilovat, slinkovat ...
- ... muset řešit binárku, která někde funguje a někde ne ...
- ... moc práce!
- (i když výhody jsou zřejmé – rychlost, distribuce binárek místo zdrojáků, “uzavřené” prostředí)
- bylo by lepší mít někdy místo motorové pily sekeru

Interpretované jazyky / skripty

- textový vstupní soubor (zdrojový kód) + interpret
- vhodné pro aplikace bez vysokých nároků na systém
- nebo tam, kde je zásadní snížit nároky na vývoj
- tj. ideální pro jednoúčelové a krátkodobě žijící programy
- většinou “volnějš” pojetí programování, z čehož plyne například řádově elegantnější práce s textem

Vlastnosti skriptovacích jazyků

Výhody

- dokonalá přenositelnost (textové vstupní soubory)
- nic se nekompiluje
- většinou syntakticky úsporné

Vlastnosti skriptovacích jazyků

Výhody

- dokonalá přenositelnost (textové vstupní soubory)
- nic se nekompile
- většinou syntakticky úsporné

Nevýhody

- zdrojový kód je otevřený (ne vždy se to hodí)
- pomalé a paměťově náročné
- bez kontroly správnosti při kompilaci

Přehled hlavních jazyků

- BAT** – vhodné pouze pro to nejjednodušší použití; i když jsou k mání některé trochu složitější funkce, jejich použití je hodně neobratné a neefektivní
- BASH** – podstatně mocnější alternativa BAT souborů v prostředí Unixu; nepříliš intuitivní syntaxe a absence náročnějších operací
- Perl** – kompaktní a efektní jazyk, který je všude nainstalovaný, ale nedá se (vůbec) číst a už i pole apod. jsou nekřesťanský obskurní
- Python** – velmi slušný jazyk, který snese i “vážnější” využití, ale za cenu trochu vyšší obtížnosti
- Ruby** – elixír síly a zázračná pilulka: intuitivní, snadný, všemocný, rozšířený a k tomu ryze objektový

Jazyk Ruby

- čistě objektový interpretovaný jazyk
- interprety existují pro širokou škálu platforem
- velmi elegantní syntaxe
- nevýhodou je stále ještě relativní pomalost
- aktuální verze 4.0 (rozumné minimum je 3.4, míň nebrat)

Ukázka Ruby (1)

Každý programátor tím začíná ...

```
puts "Hello world!"
```

Ukázka Ruby (1)

Každý programátor tím začíná ...

```
puts "Hello world!"
```

```
Hello world!
```

Ukázka Ruby (2)

Proměnné, `print` vs. `puts`, aritmetika

```
a = 4  
b = 5  
print "4 + 5 = "  
puts a + b
```

Ukázka Ruby (2)

Proměnné, `print` vs. `puts`, aritmetika

```
a = 4  
b = 5  
print "4 + 5 = "  
puts a + b
```

```
4 + 5 = 9
```

Ukázka Ruby (3)

In-line výrazy v řetězcích

```
a = 4  
b = 5  
puts "#{a} + #{b} = #{a+b}"
```

Ukázka Ruby (3)

In-line výrazy v řetězcích

```
a = 4  
b = 5  
puts "#{a} + #{b} = #{a+b}"
```

```
4 + 5 = 9
```

Důležitá vsuvka

Řetězec

"a"

Důležitá vsuvka

Řetězec

"a"

Proměnná

a

Důležitá vsuvka

Řetězec

```
"a"
```

Proměnná

```
a
```

Řetězec s proměnnou uvnitř

```
"#{a}"
```

Ukázka Ruby (4)

Rozsahy a cykly

```
(1..5).each do |i|  
  puts "#{i} * #{i} = #{i * i}"  
end
```

Ukázka Ruby (4)

Rozsahy a cykly

```
(1..5).each do |i|  
  puts "#{i} * #{i} = #{i * i}"  
end
```

```
1 * 1 = 1  
2 * 2 = 4  
3 * 3 = 9  
4 * 4 = 16  
5 * 5 = 25
```

Ukázka Ruby (5)

Pětkrát nic umožilo osla (opakování, ne cyklus)

```
5.times do  
  puts "nic"  
end
```

Ukázka Ruby (5)

Pětkrát nic umorilo osla (opakování, ne cyklus)

```
5.times do  
  puts "nic"  
end
```

```
nic  
nic  
nic  
nic  
nic
```

IRb

Pro první ozkoušení (a i pro některé úkoly v praktickém životě) se hodí “příkazová řádka” Ruby, tzv. **Interactive Ruby** (IRb):

IRb

Pro první ozkoušení (a i pro některé úkoly v praktickém životě) se hodí “příkazová řádka” Ruby, tzv. **Interactive Ruby** (IRb):

```
irb(main):001 > 2+2  
=> 4  
irb(main):002 > a = 5  
=> 5  
irb(main):003 > b = 6  
=> 6  
irb(main):004 > a * b  
=> 30
```

Proměnné, výpis na terminál

V Ruby (jak je u skriptů zvykem) se proměnné nedeklarují:

```
a = 5  
a = a * a  
long_string = "loooooong string"
```

Proměnné, výpis na terminál

V Ruby (jak je u skriptů zvykem) se proměnné nedeklarují:

```
a = 5  
a = a * a  
long_string = "loooooong string"
```

Výpis se děje pomocí `print`, resp. `puts` (bez/s koncem řádku);
`# { . . . }` vkládá do řetězce libovolný výraz:

```
print a  
puts "a = #{a}"
```

Pole a hashe

Pole je seznam:

```
a = []  
a << 5  
a += [6]  
puts a.size
```

Pole a hashe

Pole je seznam:

```
a = []  
a << 5  
a += [6]  
puts a.size
```

Hash, neboli slovník či asociativní pole:

```
b = {}  
b[3] = 7  
b["foo"] = "bar"
```

Rozsahy a cykly

Rozsahy (ranges) - se dvěma tečkami včetně posledního elementu, se třemi bez něj

```
a = (1..5)
b = (1...5)
puts "yay!" if a.size == b.size + 1
```

Rozsahy a cykly

Rozsahy (ranges) - se dvěma tečkami včetně posledního elementu, se třemi bez něj

```
a = (1..5)
b = (1...5)
puts "yay!" if a.size == b.size + 1
```

Ruby nepoužívá klasický cyklus, ale iterátor (přes téměř cokoliv):

```
(1..5).each do |i|
  puts i * i
end
b = {}; b["key1"] = 6; b["key2"] = 8
b.each do |key, value|
  puts "#{key} => #{value}"
end
```

Práce s řetězci, include, split, sub

Řetězce v Ruby jsou neomezené délky (pár mega se tam určitě vejde) a dá se s nimi provádět ledacos.

```
s = "lazy dog"
if s.include?("lazy")
  puts "lazy!!!"
end
```

Práce s řetězci, include, split, sub

Řetězce v Ruby jsou neomezené délky (pár mega se tam určitě vejde) a dá se s nimi provádět ledacos.

```
s = "lazy dog"
if s.include?("lazy")
  puts "lazy!!!"
end
```

Rozdělit? Nahradit?

```
puts s.sub("lazy", "crazy")
a = s.split
puts "#{a[1]} #{a[0]}"
```

Načítání a zápis do souboru

Soubor a terminál, to je vlastně jedno:

```
File.open("animals.txt", "w") do |f|  
  f.puts "quick brown fox"  
end
```

Načítání a zápis do souboru

Soubor a terminál, to je vlastně jedno:

```
File.open("animals.txt", "w") do |f|  
  f.puts "quick brown fox"  
end
```

Nejjednodušší čtení je po řádcích:

```
IO.foreach("data.csv") do |line|  
  ...  
end
```

Práce s adresářem

Jak projít všechny soubory v adresáři? V Pascalu utrpení,
v Ruby iterátor:

```
Dir["*"].each do |filename|  
  puts filename  
end
```

Práce s adresářem

Jak projít všechny soubory v adresáři? V Pascalu utrpení, v Ruby iterátor:

```
Dir["*"].each do |filename|  
  puts filename  
end
```

Lze použít podle očekávání libovolnou masku nebo cestu:

```
Dir["data/*.csv"].each do |filename|  
  IO.foreach(filename) do |line|  
    puts line  
  end  
end
```

Úvod
Odbočka: správa zdrojového kódu
Ruční a poloautomatická řešení
Skriptovací jazyky

Úvod do skriptování
Úvod do jazyka Ruby
První kroky s Ruby

A to je vše, přátelé!

